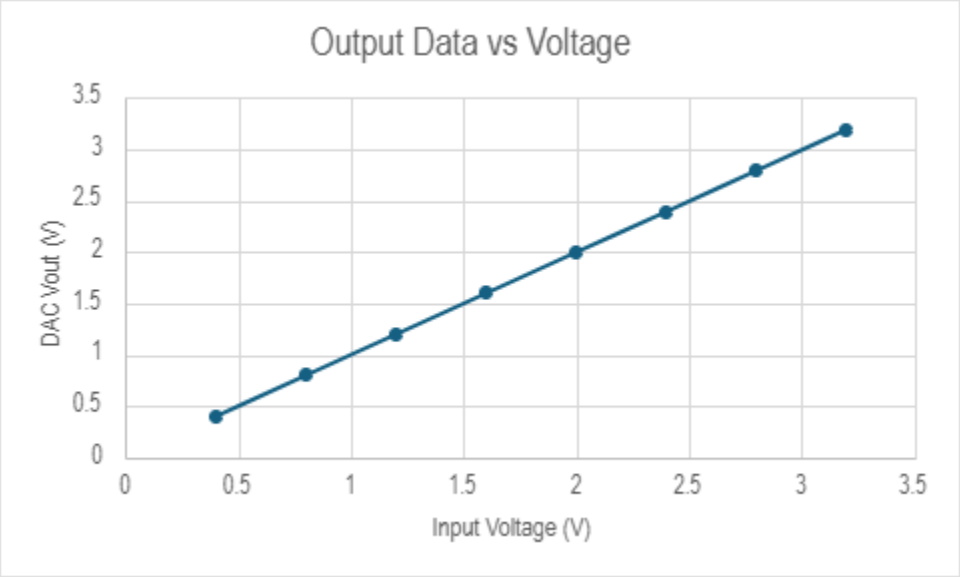
Maddie Masiello

Lab Group A5
EE 329-03 S'25
2025-May-14

Introduction:
This project demonstrates the integration of an STM32L4 microcontroller with an MCP4821 digital-to-analog converter (DAC) using the SPI communication protocol. A 4x3 matrix keypad is used as user input to enter a 3-digit voltage value, which is then converted into a 12-bit DAC word and transmitted to the MCP4821. The system enables real-time voltage output from 0.00 V to 3.30 V, with input validation, output capping, and reset functionality. The implementation was verified using a logic analyzer and calibrated to meet specified accuracy requirements.

Link to YouTube Video Presentation: https://youtu.be/RLMiT1T3Tg0
(Date of posted video: 5/15/25)
(Length: 2:00)

Obtained Data:



Calculations:
INL = Videal – Vactual = 3V-2.9913 = 0.0087 = 8.7mV
2.048/4096 = 0.5mV = expected change

Hex	Measured	DNL	LSB
0x26C	0.31332	55mV	1.1
0x266D	0.31387		
0x4D8	0.61844	51mV	1.02

0x4D9	0.61895		
0x9B1	1.23881	49mV	0.98
0x9B2	1.2393		

Figure A5(a): Wiring Diagram connecting our MCP4821 DAC with the STM32 Nucleo Board, and the 4x3 Keypad

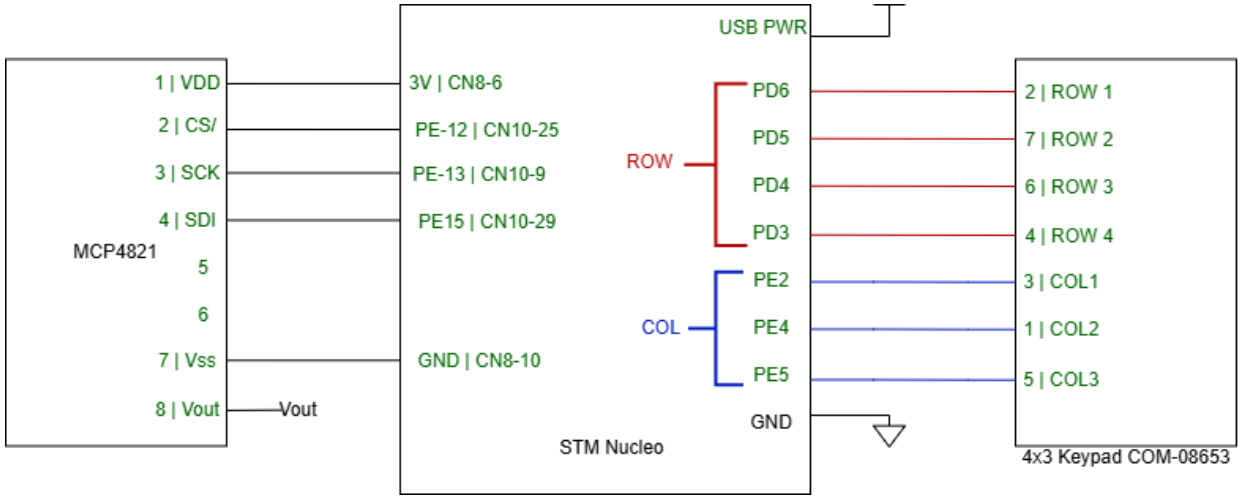
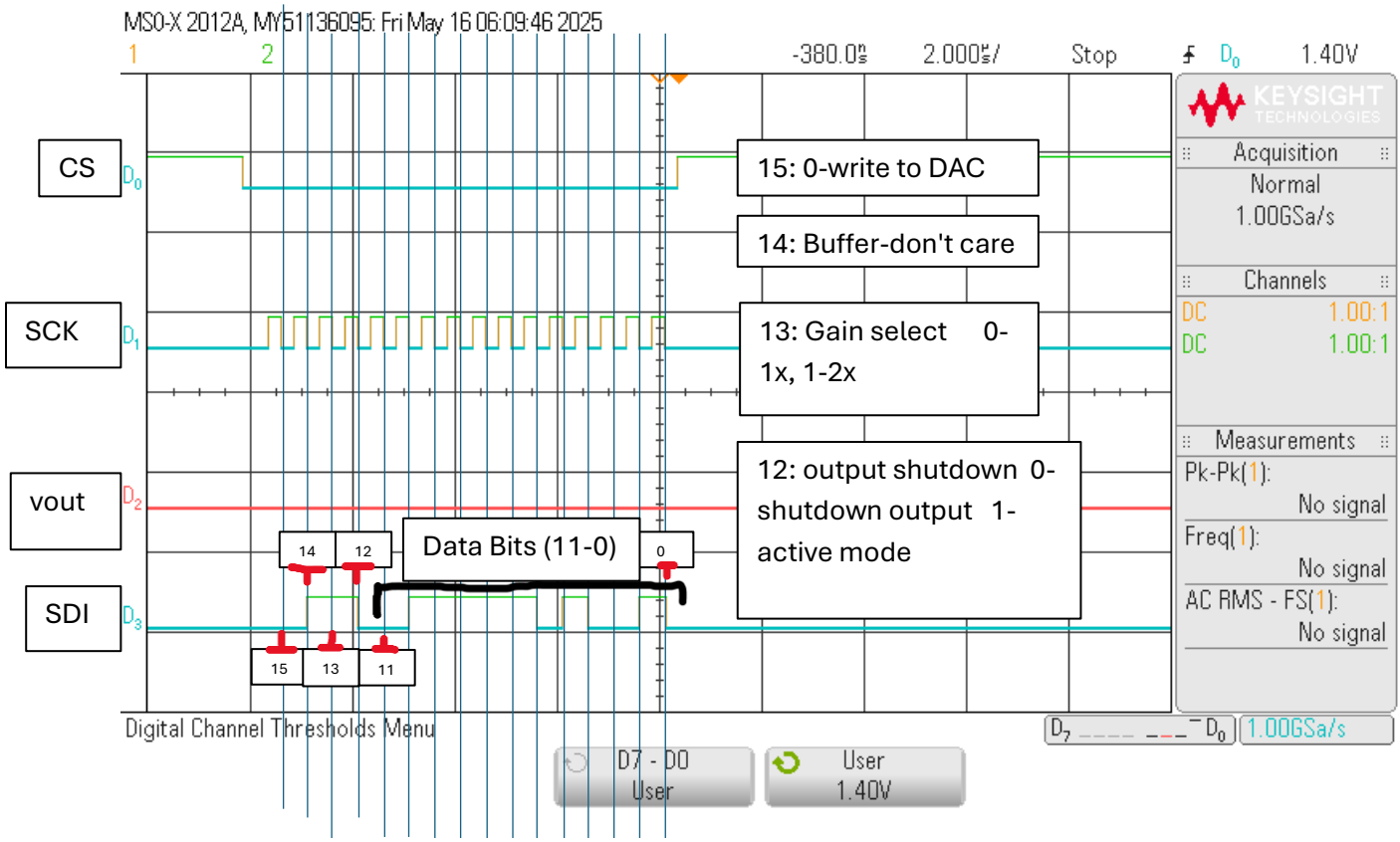
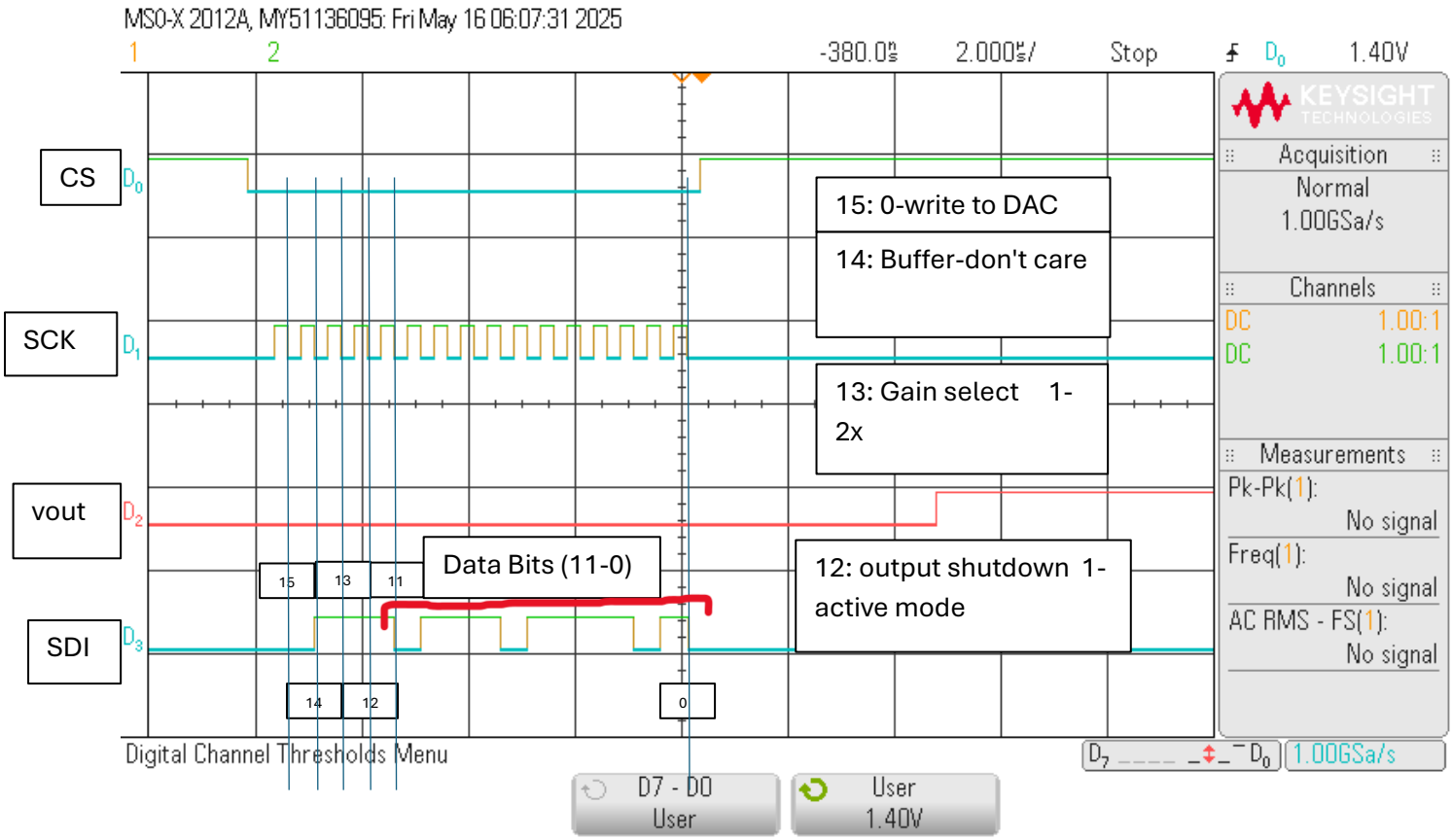


Figure A5(b): Logic Analyzer Timing Diagram for 0.5 V (top) and 1.5 V (bottom) – D0 = Clock Select, D1 = Clock, D2 = Vout, D3 = SDI.





Appendix:

Formatted Source Code:

----------main.c----------

```
/* USER CODE BEGIN Header */

/**************************************************************************
 * EE 329 A5
 **************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 * project         : EE 329 S'25 Assignment A5
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
                     Christopher Hoferer – choferer@calpoly.edu
 * version         : 1.0
 * date            : 2025-05-11
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 **************************************************************************
 * main.c implements a simple DAC interface using SPI to convert
 * millivolt values to a 16-bit DAC word. The program reads input from
 * a 4x3 matrix keypad and converts the input to a millivolt value
 * to be sent to the DAC. The keypad is debounced and the input is
 * validated to ensure that only valid millivolt values are sent to the DAC.
```

```
 * The program also includes a reset function to clear the input
 * and start over.
 ******************************************************************************/
/* USER CODE END Header */

#include "main.h"
#include "spi.h"
#include "delay.h"
#include "keypad.h"

void SystemClock_Config();

int main(void)
{
    // Initialize system clock and peripherals
    HAL_Init();
  SystemClock_Config();
  SysTick_Init();
  Keypad_Config();
  DAC_init();

    int digit_count = 0;
    int mvolt = 0;
    int key;

    while (1) {
        key = Keypad_WhichKeyIsPressed(); // Read keypad input
        if (key == NO_KEYPRESS)
            continue;

        if (key == 10) { // '*' resets input
            digit_count = 0;
            mvolt = 0;
            DAC_write(0);  // Reset DAC output to 0V
            continue;
        }

        if (key >= 0 && key <= 9) {
            if (digit_count < 3) {
                mvolt = mvolt * 10 + key;
                digit_count++;
            }

        if (digit_count == 3) {
            mvolt *= 10; // Scale to millivolts (e.g., 123 → 1230 mV)
            if (mvolt > 3300) {
                mvolt = 3300;
            }
            DAC_write(mvolt);
            digit_count = 0;
```

```
            mvolt = 0;
            delay_us(100000); // Debounce
        }
    }
}

}

// Initialize DAC by initializing SPI
void DAC_init(void) {
    SPI_init();
}

// Write millivolt value to DAC
void DAC_write(int mvolts) {
    uint16_t data = DAC_volt_conv(mvolts);

    while (!(SPI1->SR & SPI_SR_TXE));  // Wait for transmit complete
    SPI1->DR = data;
    while (SPI1->SR & SPI_SR_BSY);     // Wait until not busy
}

// Convert millivolts to 16-bit DAC word with control bits
int DAC_volt_conv(int mvolts) {
    //mvolts = (mvolts * 10000 / 9981) + 1;  // update with calibrated slope

    mvolts = (mvolts * 10000 / 9981) + 1;  //mV

    uint16_t data;
    //if millivolts is less than 2.048V then gain = 1
    //if millivolts is between 2.048V and 3.3V then gain = 2
    //if millivolts is greater than 3.3V then max value
    if (mvolts <= 2048) {
        data = (mvolts * 4095) / 2048;
        return (data | (0x3 << 12));  // Gain = 1
    } else if (mvolts <= 3300) {
        data = (mvolts * 4095) / (2 * 2048);
        return (data | (0x1 << 12));  // Gain = 2
    } else {
        return 0x1FFF;  // Max value
    }
}
/**************************************************************************
 * System Configuration
 **************************************************************************/
/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
```

```c
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
  RCC_OscInitStruct.MSIState = RCC_MSI_ON;
  RCC_OscInitStruct.MSICalibrationValue = 0;
  RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
```

```
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

----------main.h----------

```
/******************************************************************************

 * EE 329 A5
 ******************************************************************************
 * @file           : main.h
 * @brief          :
 * project         : EE 329 S'23 Assignment A5
 * authors         : Maddie Masiello
                     Christopher Hoferer - choferer@calpoly.edu
 * version         : 1
 * date            : 230413
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 *
 ******************************************************************************
 * REVISION HISTORY
 * 230413 - Masiello - initial version
 ******************************************************************************
 * TODO
 *
```

```
 ******************************************************************************
 */

 #ifndef MAIN_H
 #define MAIN_H

 #include "stm32l4xx.h"
 #include "stm32l4xx_hal.h"

 // Function Prototypes
 void SystemClock_Config(void);
 void Error_Handler(void);
 void DAC_init(void);
 void DAC_write(int mvolts);
 int  DAC_volt_conv(int mvolts);

 #endif
```

----------spi.c----------

```
/* USER CODE BEGIN Header */
/******************************************************************************
 * EE 329 A5
 ******************************************************************************
 * @file           : spi.c
 * @brief          : SPI configuration
 * project         : EE 329 S'23 Assignment A5
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
                     Christopher Hoferer – choferer@calpoly.edu
 * version         : 0.1
 * date            : 5/12/25
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * spi.c implements SPI communication for the MCP4821 DAC
 * REVISION HISTORY
 * 0.1
 ******************************************************************************
 */

#include "spi.h"
#include "stm32l4xx_hal.h"

#include "spi.h"

// Define SPI port and bitmask groups
#define SPIPORT GPIOE
#define SPI_RCC RCC_AHB2ENR_GPIOEEN
```

```
#define SPI_MODER (GPIO_MODER_MODE12 | GPIO_MODER_MODE13 | GPIO_MODER_MODE14 |
GPIO_MODER_MODE15)
#define SPI_MODER_AF (GPIO_MODER_MODE12_1 | GPIO_MODER_MODE13_1 | GPIO_MODER_MODE14_1 |
GPIO_MODER_MODE15_1)
#define SPI_OSPEEDR ((3 << GPIO_OSPEEDR_OSPEED12_Pos) | (3 << GPIO_OSPEEDR_OSPEED13_Pos) | \
                     (3 << GPIO_OSPEEDR_OSPEED14_Pos) | (3 << GPIO_OSPEEDR_OSPEED15_Pos))
#define SPI_AFRCLEAR ((0xF << GPIO_AFRH_AFSEL12_Pos) | (0xF << GPIO_AFRH_AFSEL13_Pos) | \
                      (0xF << GPIO_AFRH_AFSEL14_Pos) | (0xF << GPIO_AFRH_AFSEL15_Pos))
#define SPI_AFRSET ((0x5 << GPIO_AFRH_AFSEL12_Pos) | (0x5 << GPIO_AFRH_AFSEL13_Pos) | \
                    (0x5 << GPIO_AFRH_AFSEL14_Pos) | (0x5 << GPIO_AFRH_AFSEL15_Pos))

void SPI_init(void) {
    // enable clock for GPIOA & SPI1
    RCC->AHB2ENR |= SPI_RCC;                    // GPIOE: DAC NSS/SCK/SDO
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;         // SPI1 port

    // configure AF select, push pull, no pu/pd/ fast mode
    SPIPORT->MODER   &= ~SPI_MODER;     // clear MODER
    SPIPORT->MODER   |=  SPI_MODER_AF;  // set AF mode
    SPIPORT->OSPEEDR |=  SPI_OSPEEDR;   // set fast speed
    SPIPORT->AFR[1]  &= ~SPI_AFRCLEAR;  // clear AF bits
    SPIPORT->AFR[1]  |=  SPI_AFRSET;    // set AF bits

    // SPI config as specified @ STM32L4 RM0351 rev.9 p.1459
    // build control registers CR1 & CR2 for SPI control of peripheral DAC
    // assumes no active SPI xmits & no recv data in process (BSY=0)
    // CR1 (reset value = 0x0000)
    SPI1->CR1 &= ~( SPI_CR1_SPE );              // disable SPI for config
    SPI1->CR1 &= ~( SPI_CR1_RXONLY );           // recv-only OFF
    SPI1->CR1 &= ~( SPI_CR1_LSBFIRST );         // data bit order MSb:LSb
    SPI1->CR1 &= ~( SPI_CR1_CPOL | SPI_CR1_CPHA ); // SCLK polarity:phase = 0:0
    SPI1->CR1 |=     SPI_CR1_MSTR;                  // MCU is SPI controller
    // CR2 (reset value = 0x0700 : 8b data)
    SPI1->CR2 &= ~( SPI_CR2_TXEIE | SPI_CR2_RXNEIE ); // disable FIFO intrpts
    SPI1->CR2 &= ~( SPI_CR2_FRF);               // Moto frame format
    SPI1->CR2 |=     SPI_CR2_NSSP;                  // auto-generate NSS pulse
    SPI1->CR2 |=     SPI_CR2_DS;                    // 16-bit data
    SPI1->CR2 |=     SPI_CR2_SSOE;                  // enable SS output
    // CR1
    SPI1->CR1 |=     SPI_CR1_SPE;                   // re-enable SPI for ops
}
```

----------spi.h----------

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A5
 **************************************************************************
```

```
 * @file           : spi.h
 * @brief          : header file for SPI configuration and communication funcs
 * project         : EE 329 S'23 Assignment A5
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
                     Christopher Hoferer – choferer@calpoly.edu
 * version         : 0.1
 * date            :
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics. All rights reserved.
 ******************************************************************************
 *
 * REVISION HISTORY
 * 0.1
 ******************************************************************************
 */
#ifndef SPI_H
#define SPI_H

#include "stm32l4xx.h"

void SPI_init(void);

#endif // SPI_H
```

----------keypad.h----------

```
/* USER CODE BEGIN Header */
/******************************************************************************
 * EE 329 A2 KEYPAD INTERFACE
 ******************************************************************************
 * @file           : keypad.h
 * @brief          : Keypad GPIO pin mapping and constants for a 4x3 matrix keypad
 * project         : EE 329 S'23 Assignment 2
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
                     Christopher Hoferer – choferer@calpoly.edu
 * version         : 0.1
 * date            :
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * KEY MAPPING:
 * Column outputs: GPIOE pins PE2 (COL1), PE4 (COL2), PE5 (COL3)
 * Row inputs:     GPIOD pins PD6 (ROW1), PD5 (ROW2), PD4 (ROW3), PD3 (ROW4)
 *
 * MATRIX LOGIC:
```

```
 *     COL1  COL2  COL3
 * R1   1     2     3
 * R2   4     5     6
 * R3   7     8     9
 * R4   *     0     #
 *
 * Encoded key values: 1-9 as-is, '*'=10, '0'=11, '#'=12
 * For LED display, key '0' is remapped to 0xF (CODE_ZERO)
 *
 * REVISION HISTORY
 * 0.1
 * 0.2 cwh added debounce
 ******************************************************************************
 */

#ifndef INC_KEYPAD_H_
#define INC_KEYPAD_H_

#include "stm32l4xx_hal.h"

// ------------------------ Boolean Constants ------------------------
#define TRUE 1
#define FALSE 0

// ------------------------ Keypad GPIO Mapping ------------------------
// COL_PORT = GPIOE: PE2 (COL1), PE4 (COL2), PE5 (COL3)
// ROW_PORT = GPIOD: PD6 (ROW1), PD5 (ROW2), PD4 (ROW3), PD3 (ROW4)

#define COL_PORT GPIOE
#define ROW_PORT GPIOD

// Columns (in logical order: COL1, COL2, COL3)
#define COL_PINS (GPIO_PIN_2 | GPIO_PIN_4 | GPIO_PIN_5)
#define BIT0_COL GPIO_PIN_2 // First bit for column scanning

// Rows (in logical order: ROW1, ROW2, ROW3, ROW4)
#define ROW_PINS (GPIO_PIN_6 | GPIO_PIN_5 | GPIO_PIN_4 | GPIO_PIN_3)
#define BIT0_ROW GPIO_PIN_6 // First bit for row scanning (ROW1 = PD6)

// Compatibility define for legacy use
#define BIT0 BIT0_ROW

// ------------------------ Keypad Geometry ------------------------
#define NUM_COLS 3
#define NUM_ROWS 4

// ------------------------ Timing ------------------------
#define SETTLE 1900 // ~20ms settle delay for signal to stabilize

// ------------------------ Keypad Key Encoding ------------------------
```

```
// Encoded values: 1–9 = keys 1–9
// '*' = 10, '#' = 12, '0' = 11 → remapped to 0xF for LED
#define NO_KEYPRESS -1
#define KEY_ZERO 11
#define CODE_ZERO 0xF // 0xF = 15 → 4-bit LED all ON for '0'

#define LOOP_PER_DELAY 395.75 // 1ms delay = 395.75 loops at 4MHz

// ------------------------ Function Prototypes ------------------------
void Keypad_Config(void);
int Keypad_DebouncedKey(void);
void Keypad_UpdateKey(void);
uint8_t keypad_Debounce( int row);
int Keypad_WhichKeyIsPressed(void);


#endif /* INC_KEYPAD_H_ */
/* Function to handle 3-key voltage input */
float Keypad_HandleVoltageInput(void);
```

----------keypad.c----------

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 5 KEYPAD INTERFACE
 **************************************************************************
 * @file           : keypad.c
 * @brief          : keypad configuration and debounced detection of keypresses
 * project         : EE 329 S'23 Assignment A5
 * authors         : Maddie Masiello
                     Christopher Hoferer – choferer@calpoly.edu
 * version         : 0.1
 * date            : 5/12/2025
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics.  All rights reserved.
 **************************************************************************
 * Keypad Wiring 4 ROWS 3 COLS (pinout NUCLEO-L4A6ZG = L496ZG)
 **************************************************************************
 * REVISION HISTORY
 * 0.1
 * 0.2 cwh added debounce
 **************************************************************************
 */

#include "keypad.h"
#include "delay.h"
```

```c
int current_key = NO_KEYPRESS;
int last_key = NO_KEYPRESS;
int last_key_pressed_time = -1;

/* ---------------------------- Keypad_Config()------------------------------
 * Keypad_Config() initializes the GPIO pins for the keypad interface.
 * The keypad is a 4x3 matrix with 4 rows and 3 columns.
 * The rows are configured as inputs with pull-down resistors,
 * and the columns are configured as outputs.
 * The function enables the GPIOF clock and sets the appropriate mode, type,
 * speed, and pull-up/pull-down settings for the pins.
 * ------------------------------------------------------------------------- */
void Keypad_Config(void)
{
   // Enable Clocks
   __HAL_RCC_GPIOE_CLK_ENABLE();
   __HAL_RCC_GPIOD_CLK_ENABLE();

   // Configure PE2, PE4, PE5 (Columns) as Output
   GPIO_InitTypeDef GPIO_InitStruct = {0};
   GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
   GPIO_InitStruct.Pull = GPIO_NOPULL;
   GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

   GPIO_InitStruct.Pin = GPIO_PIN_2 | GPIO_PIN_4 | GPIO_PIN_5;
   HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

   // Configure PD3, PD4, PD5, PD6 (Rows) as Input with Pull-Down
   GPIO_InitStruct.Pin = GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6;
   GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
   GPIO_InitStruct.Pull = GPIO_PULLDOWN;
   HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
}

/* ---------------------------- Keypad_IsAnyKeyPressed()----------------------
 * Keypad_IsAnyKeyPressed() checks if any key on the keypad is pressed.
 * It drives all columns high and checks if any row is high.
 * If a key is pressed, it returns TRUE; otherwise, it returns FALSE.
 * No debounce, just looking for a key twitch.
 * ------------------------------------------------------------------------- */
int Keypad_IsAnyKeyPressed(void)
{
   // drive all COLUMNS HI; see if any ROWS are HI
   // return true if a key is pressed, false if not
   // currently no debounce here - just looking for a key twitch
   COL_PORT->BSRR = COL_PINS;           // set all columns HI
   if ((ROW_PORT->IDR & ROW_PINS) != 0) // got a keypress!
      return TRUE;
   else
      return FALSE;
```

```
}

/* --------------------------- Keypad_WhichKeyIsPressed()--------------------
 * Keypad_WhichKeyIsPressed() detects and encodes a pressed key at {row,col}
 * It assumes a previous call to Keypad_IsAnyKeyPressed() returned TRUE.
 * It verifies the Keypad_IsAnyKeyPressed() result (no debounce here),
 * determines which key is pressed, and returns the encoded key ID.
 * ------------------------------------------------------------------------ */
int Keypad_WhichKeyIsPressed(void)
{
   int8_t iRow = 0, iCol = 0;
   int8_t bGotKey = 0;
   int iKey = NO_KEYPRESS;

   const uint16_t colMap[NUM_COLS] = {GPIO_PIN_2, GPIO_PIN_4, GPIO_PIN_5};
   const uint16_t rowMap[NUM_ROWS] = {GPIO_PIN_6, GPIO_PIN_5, GPIO_PIN_4, GPIO_PIN_3};

   // Step 1: Set all columns high
   COL_PORT->BSRR = COL_PINS;

   for (iRow = 0; iRow < NUM_ROWS; iRow++)
   {
      if (ROW_PORT->IDR & rowMap[iRow])
      {
         // Step 2: Pull all columns low
         COL_PORT->BRR = COL_PINS;

         // Step 3: Raise one column at a time
         for (iCol = 0; iCol < NUM_COLS; iCol++)
         {
            COL_PORT->BSRR = colMap[iCol]; // Raise one column
            delay_us(50);                  // Let signals settle

            if (ROW_PORT->IDR & rowMap[iRow])

            {
               bGotKey = keypad_Debounce(rowMap[iRow]);
               break;
            }

            COL_PORT->BRR = colMap[iCol]; // Reset column
         }
         if (bGotKey)
            break;
      }
   }

   if (bGotKey)
   {
      iKey = (iRow * NUM_COLS) + iCol + 1;
```

```
      if (iKey == KEY_ZERO)
         iKey = CODE_ZERO;
   }

   return iKey;
}
```

```
/* ---------------------------- Keypad_DebouncedKey() ----------------------- */
int Keypad_DebouncedKey(void)
{
   // Call Keypad_UpdateKey() to check for keypresses
   Keypad_UpdateKey();
   delay_us(400);
   return current_key;
}

uint8_t keypad_Debounce( int row) {
   // number of valid checks
   uint16_t valid_check = 0;
   // loop for number of iterations to make 20ms delay
   for ( uint16_t i = 0; i < ( LOOP_PER_DELAY*20 ); i++ ) {
      if ( ( ROW_PORT->IDR & row ) != 0 ) { // if pin is high
         valid_check++;                                  // add to num of vld chks
      }
   }
   // if the number of valid checks in that time exceeds 60% of the time
   // return a key press (1)
   if ( valid_check > ( .5*LOOP_PER_DELAY*20 ) ) {
      return 1;
   }
   // otherwise return no key press (0)
   return 0;

}
```

----------delay.h----------

```
#ifndef INC_DELAY_H_
#define INC_DELAY_H_

#include "stm32l4xx.h"

void SysTick_Init(void);
void delay_us(uint32_t time_us);
```

```
#endif /* INC_DELAY_H_ */
```

----------delay.c----------

```
#include "delay.h"

void SysTick_Init(void)
{
    SysTick->CTRL |= (SysTick_CTRL_ENABLE_Msk | SysTick_CTRL_CLKSOURCE_Msk);
    SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;
}

void delay_us(uint32_t us)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000);
    while ((DWT->CYCCNT - start) < ticks)
        ;
}
```